Sharing Objects With let

RSpec gives us an alternative construct, let, which handles this edge case. It gives us a nicer, less chatty syntax as well. Remove your sandwich method and replace it with the following code:

01-getting-started/09/spec/sandwich_spec.rb
```
let(:sandwich) { Sandwich.new('delicious', []) }
```

You can think of let as binding a name (sandwich) to the result of a computation (the block). Just as with a memoized helper method, RSpec will run the block the first time any example calls sandwich.

When you run your specs again with your let definition, they should still pass.

It's possible to go overboard in our quest to reduce duplication. We can end up with test suites that we can only read by bouncing back and forth endlessly between the examples and let declarations. Our recommendation is to use these code-sharing techniques where they improve maintainability, lessen noise, and increase clarity.

Your Turn

In this chapter, you installed RSpec and took it for a test drive. You wrote a few simple specs to get a feel for the major parts of the framework. You saw how to run your examples and interpret the output. Finally, you explored a few different ways to reduce duplication in your specs.

Now, it's time to put this knowledge to the test.

Exercises

1. We've shown you three primary ways to reduce duplication in RSpec: hooks, helper methods, and let declarations. Which way did you like best for this example? Why? Can you think of situations where the others might be a better option?

2. Run rspec --help and look at the available options. Try using a few of them to run your sandwich examples.

Ready to see some of our favorite ways to use RSpec? Take a quick sandwich break and then meet us in the next chapter.